

AI Engineering & Generative AI — Professional Masterclass

Build production-grade AI applications, not just prompts.

This is an original, project-first professional course manual created for educational and commercial course-development use.

The material is designed to be taught, adapted, branded and expanded. It emphasizes practical engineering, portfolio projects, assessments and production thinking rather than certificate collection. Technology names are used for educational context; specific provider pricing, product limits and interfaces should be verified before publication because they change over time.

Course Architecture

- 7 large modules with lessons, practical exercises, assessments and projects.
- Recommended delivery: 10–14 weeks at 6–10 hours per week, or self-paced.
- Primary outcome: a portfolio-ready capstone plus reusable professional skills.
- Suggested teaching method: concept → demonstration → guided lab → independent project → assessment.

Module Roadmap

1	AI Engineering Foundations
2	LLM Application Architecture
3	Embeddings, RAG and Knowledge Systems
4	Tool Calling, Agents and MCP
5	Fine-Tuning, Multimodal AI and Advanced Models
6	Evaluation, Safety, Security and Production Operations
7	Capstone: Production AI SaaS

Module 1 — AI Engineering Foundations

Learning objectives

- What AI engineering actually is
- Python and software foundations for AI products
- APIs, SDKs, environments and dependency management
- Tokens, embeddings, context windows and model behavior
- Model selection: capability, latency, cost and reliability
- Prompt design as an engineering discipline
- Mini-project: build a model-powered command-line assistant

Core lesson

AI engineering is the discipline of turning foundation models and machine-learning capabilities into dependable software. A useful AI engineer does not stop at getting a model to answer a question. The engineer designs inputs, controls context, validates outputs, handles failures, measures quality, protects data and connects the model to a real user workflow.

Start with the software foundation. Use Python for rapid AI experimentation, but apply normal engineering practices: virtual environments, package pinning, configuration files, logging, exception handling, type hints, tests and Git. Keep secrets out of source code. Separate business logic from model calls so that a provider can be replaced without rewriting the application.

A language model predicts tokens from context. That simple statement explains many practical behaviors. The model does not automatically know whether a response is true, current or appropriate. The application therefore needs a system around the model. That system can include retrieval, tools, structured output validation, safety checks, caching, evaluation and human review.

Learn the vocabulary early. A token is a unit used by the model's tokenizer. A context window is the amount of input and generated output the model can consider in one interaction. An embedding is a numerical representation that allows semantic similarity search. Temperature and related sampling settings influence variability. Latency is the time to produce a response, while throughput describes how much work a system can process.

Prompting should be treated like interface design. Define the role, task, constraints, input format and desired output. Prefer explicit instructions and examples over vague requests. When a task has a stable structure, ask for structured JSON or another schema and validate it before using it downstream.

A production architecture commonly looks like: client → API → authentication → application service → model gateway → tools/retrieval → validation → persistence → observability. Build the smallest version first, then add components when a measured problem requires them.

Practical lab

- Set up a clean Python project with a virtual environment and environment variables.
- Create a model gateway with one function such as `generate(request)`.
- Write prompts for three versions of the same task and compare consistency.
- Record latency, token usage or approximate cost for repeated requests.
- Add retries with bounded backoff and a clear timeout policy.

Assessment

Assessment: Explain why a model response cannot be treated as automatically correct. Design a small AI service architecture and identify five possible failure points.

Module project

Project: AI Command Assistant — accept a user task, call a model, return a structured response, log request metadata, and handle API failures gracefully.

Module 2 — LLM Application Architecture

Learning objectives

- Model APIs and provider abstraction
- Conversation state and memory
- Structured outputs and schema validation
- Streaming responses and user experience
- Caching, rate limits and retries
- Cost and latency engineering
- Mini-project: production-ready AI chat API

Core lesson

A serious LLM application needs an architecture that controls the model rather than allowing the model to control the architecture. Begin with a provider abstraction. Your application should call a stable internal interface while the implementation decides which model or provider to use. This makes experimentation easier and reduces vendor lock-in.

Conversation state should be explicit. Distinguish short-term conversation history from durable user preferences and application data. Do not place unlimited history into every request. Summarize old turns, retrieve only relevant memories, and keep authoritative facts in a database rather than relying on the model's memory.

Structured outputs are valuable when model responses feed software. Define a schema such as fields, types, allowed values and required properties. Validate the response before it reaches business logic. If validation fails, retry with a corrected instruction or route the task to a fallback. This is safer than parsing arbitrary prose with fragile string operations.

Streaming is primarily a user-experience technique. Instead of waiting for the entire generation, the server can send partial output. However, streaming introduces complexity: cancellation, reconnects, incomplete output and moderation timing. Design these cases deliberately.

Reliability needs several layers. Set timeouts. Retry only transient failures. Use exponential backoff with a maximum number of attempts. Add rate limiting to protect your service. Cache deterministic or expensive operations where appropriate. Track model usage and estimate cost by user, endpoint and feature.

Latency is often improved more effectively by reducing unnecessary context than by blindly selecting a faster model. Parallelize independent retrieval or tool calls, use smaller models for classification and routing, and reserve expensive models for tasks that actually need them.

Practical lab

- Design a provider-neutral model interface.
- Create a JSON schema for a support-ticket classifier.
- Implement request timeout and exponential backoff.
- Build a simple response cache for identical safe requests.
- Measure p50 and p95 latency for your endpoint.

Assessment

Assessment: Design an API that supports streaming, retries, structured output and model fallback without duplicating business logic.

Module project

Project: AI Chat API — authentication, conversation persistence, streaming, structured metadata, retry handling, usage logging and a clean provider abstraction.

Module 3 — Embeddings, RAG and Knowledge Systems

Learning objectives

- Why retrieval is needed
- Embedding models and semantic similarity
- Chunking strategies
- Metadata and filtering
- Vector databases
- Hybrid search and reranking
- Grounded generation and citations

Core lesson

Retrieval-augmented generation, or RAG, gives a model access to external knowledge at inference time. Instead of expecting the model to memorize every document, your system searches a knowledge base and supplies relevant passages to the generation step.

A typical pipeline is ingestion → cleaning → chunking → embedding → indexing → retrieval → optional reranking → prompt construction → generation → citation or source presentation. Each stage affects quality. Poorly extracted documents cannot be rescued by a sophisticated model.

Chunking is a major design decision. Very small chunks may lose context; very large chunks may contain irrelevant material and waste context. Start with semantic boundaries such as headings and paragraphs, then use a target size and controlled overlap. For structured documents, preserve metadata such as title, section, date, product, permission scope and source URL.

Vector search retrieves by semantic similarity. But semantic similarity alone may miss exact identifiers, names or numbers. Hybrid retrieval combines keyword and vector signals. Reranking can improve precision by scoring a smaller candidate set with a stronger model.

Permission-aware retrieval is essential in real products. Every document or chunk should carry access metadata. The retrieval layer must enforce authorization rather than relying on the prompt to tell the model what it may see.

Grounding is more than attaching a citation after generation. The system should instruct the model to use retrieved evidence, identify uncertainty, and avoid inventing unsupported facts. Evaluation should measure retrieval recall, answer correctness, citation correctness and refusal behavior when evidence is insufficient.

Practical lab

- Build an ingestion pipeline for Markdown or PDF text.
- Compare three chunking strategies on the same documents.
- Add metadata filtering for user or organization ownership.
- Implement top-k vector retrieval and inspect failures manually.
- Require source references in generated answers.

Assessment

Assessment: Explain the difference between retrieval quality and generation quality. Describe three RAG failure modes and their fixes.

Module project

Project: Knowledge Copilot — upload documents, index them, retrieve relevant sections, answer questions with source references, and enforce document permissions.

Module 4 — Tool Calling, Agents and MCP

Learning objectives

- From chatbots to tool-using systems
- Function/tool schemas
- Planning and execution loops
- Agent state and checkpoints
- MCP concepts and tool interoperability
- Human approval and dangerous actions
- Building a bounded agent

Core lesson

Tool use changes an LLM application from a text generator into a system that can act. A tool is an explicit capability such as search, database lookup, calculator, ticket creation or calendar operation. The model should never receive uncontrolled access to arbitrary system functions.

Define each tool with a narrow purpose, typed arguments, clear descriptions and authorization requirements. Validate arguments before execution. Return structured results. Keep tool execution outside the model and log every call.

An agent loop commonly consists of observe → decide → call tool → inspect result → continue or finish. The loop needs boundaries: maximum steps, time limits, token budgets and allowed tools. Without these controls, a poorly planned task can become expensive or unpredictable.

Agent state should be durable when tasks span multiple turns or long-running workflows. Store checkpoints so that execution can resume after failure. For actions with side effects, introduce approval gates. Reading information may be automatic while sending money, deleting records or contacting customers should require stronger controls.

The Model Context Protocol, or MCP, provides a standardized way for AI applications to connect models with tools and contextual resources. Learn it as an interoperability pattern, not as a substitute for authorization or application architecture.

The best agents are often bounded workflows rather than fully autonomous systems. Use deterministic code for deterministic steps and models for judgment-heavy steps. This reduces cost and increases reliability.

Practical lab

- Create three safe tools with JSON schemas.
- Implement an agent loop with a maximum step count.
- Add an approval checkpoint before a simulated destructive action.
- Log every tool call and its arguments/result summary.
- Design an MCP-style tool catalog for your project.

Assessment

Assessment: Explain why unrestricted autonomous agents are risky. Design an agent with explicit tool permissions, step limits and approval gates.

Module project

Project: Research Agent — search a controlled knowledge source, gather evidence, produce a cited report, and stop after a bounded number of tool calls.

Module 5 — Fine-Tuning, Multimodal AI and Advanced Models

Learning objectives

- When prompting is enough
- Supervised fine-tuning concepts
- Dataset design and quality
- Evaluation before and after tuning
- Vision and document understanding
- Speech and audio pipelines
- Model routing and ensembles

Core lesson

Fine-tuning should solve a demonstrated problem, not be used because it sounds advanced. First improve the prompt, retrieval, tools and output validation. Fine-tuning becomes attractive when you have a stable task, a representative dataset and a measurable behavior you want to change.

Dataset quality matters more than raw quantity. Remove contradictory examples, sensitive data that should not be used, duplicates and ambiguous labels. Keep a held-out evaluation set that is never used for training. Compare the tuned system against a baseline using the same tests.

Multimodal systems accept or produce combinations of text, images, audio and other data. A document-processing pipeline might extract a scanned page, identify layout, read tables, classify fields and produce structured records. Vision models can support inspection, chart understanding and image-grounded assistants, but outputs still need validation.

Speech systems commonly combine speech-to-text, reasoning and text-to-speech. Latency becomes especially important for voice applications. Stream audio where supported, keep prompts concise, and design interruption handling.

Model routing is an advanced optimization. A small model can handle classification, extraction or simple transformation while a larger model handles difficult reasoning. A router can select a model based on task complexity, cost budget or latency requirements. Evaluate routing errors because a cheap wrong answer can cost more than a slower correct one.

Practical lab

- Create a labeled dataset for a narrow classification task.
- Define an evaluation set before changing the model.
- Prototype a document-to-JSON multimodal pipeline.
- Compare small and large models on the same task.
- Design a model-routing policy based on complexity.

Assessment

Assessment: Give three situations where fine-tuning is appropriate and three where RAG or better prompting is preferable.

Module project

Project: Multimodal Document Processor — accept an image/PDF page, extract structured information, validate it and produce an audit trail.

Module 6 — Evaluation, Safety, Security and Production Operations

Learning objectives

- AI evaluation design
- Golden datasets and regression tests
- Hallucination and groundedness checks
- Prompt injection and data leakage
- Authentication and authorization
- Observability and incident response
- Production readiness checklist

Core lesson

Evaluation is the difference between an AI demo and an engineered product. Build a small golden dataset containing representative inputs, difficult edge cases and known failure cases. Run it whenever prompts, models, retrieval settings or tool behavior changes.

Measure more than a single score. Depending on the application, track task success, factual correctness, groundedness, citation accuracy, refusal correctness, latency, cost and user feedback. Human evaluation is expensive but valuable for nuanced tasks. Automated evaluators are useful for regression testing but should themselves be validated.

Security threats include prompt injection, sensitive-data leakage, insecure tool execution, broken authorization and malicious documents. Treat retrieved text and tool outputs as untrusted input. A document can contain instructions designed to manipulate the model. Keep system policies separate, validate tool arguments and enforce permissions in code.

Observability should connect user requests to model calls, retrieval operations and tool executions. Record safe metadata, not secrets or unnecessary personal content. Create dashboards for errors, latency and cost. Define incident procedures before an incident occurs.

Production readiness includes load testing, fallback behavior, backups, rate limiting, secret management, dependency updates and rollback procedures. AI systems also need a policy for uncertain outputs. Decide when to ask the user for clarification, when to abstain and when to route to a human.

Practical lab

- Create a 50-case evaluation set for your project.
- Write tests for prompt-injection resistance.
- Add authorization checks to every data/tool path.
- Create a cost and latency dashboard.
- Write a one-page incident response runbook.

Assessment

Assessment: Threat-model an AI application and list controls for prompt injection, data leakage, unauthorized tools and unreliable outputs.

Module project

Project: AI Reliability Lab — build an evaluation harness, security test suite, metrics dashboard and release checklist.

Module 7 — Capstone: Production AI SaaS

Learning objectives

- Product discovery and AI use-case selection
- Architecture and database design
- RAG/agent integration
- Authentication, billing and entitlements
- Testing and evaluation
- Deployment and monitoring
- Launch, pricing and portfolio strategy

Core lesson

The capstone combines the course into a real product. Start with a narrow user problem where AI creates measurable value. Avoid building a generic chatbot. Examples include document analysis for a specific profession, a support copilot, an internal knowledge assistant or an automated research workflow.

Write the product contract first: target user, problem, input, output, success metric, failure behavior and pricing assumption. Design the database and authorization model before implementing features. Separate user identity, organizations, subscriptions, usage records, content and AI execution logs.

Build an MVP with one core workflow. Add retrieval or tools only where they improve that workflow. Create evaluation cases before launch. Test malformed inputs, missing documents, slow model responses, revoked subscriptions and unauthorized access.

For monetization, measure the cost of serving a typical customer. Pricing must cover model usage, storage, infrastructure, payment fees, support and a margin. Usage limits are useful, but they should map to customer value rather than arbitrary technical limits.

A strong launch asset is a working demo plus a technical case study. Explain the problem, architecture, evaluation method, measured latency/cost, security controls and lessons learned. This is more persuasive to employers and clients than a certificate alone.

Practical lab

- Choose one narrow customer problem.
- Create a one-page product specification.
- Build the smallest end-to-end workflow.
- Add evaluation and security gates before launch.
- Deploy and collect real user feedback.

Assessment

Assessment: Present your product architecture, evaluation evidence, unit economics and launch plan.

Module project

Capstone: Production AI SaaS — a deployable application with authentication, a core AI workflow, retrieval/tools where appropriate, billing or usage controls, observability, tests and documentation.

Instructor & Commercial Packaging Guide

- Turn each module into 3–6 video lessons plus one guided lab.
- Provide a downloadable starter repository and a separate completed reference implementation.
- Add quizzes after each lesson and a practical assessment after each module.
- Keep project requirements stable while allowing learners to choose their own domain.
- For commercial delivery, add your own branding, instructor examples, screenshots, exercises and community/support layer.
- Avoid promising jobs, income or certification unless you actually provide the relevant accreditation or service.

Suggested Course Completion Certificate Criteria

- Complete at least 5 of 7 module projects.
- Pass the capstone assessment.
- Submit a working portfolio artifact and technical documentation.
- Demonstrate the ability to explain design decisions and limitations.

Final Portfolio Checklist

- Public or private demo URL where appropriate.
- Architecture diagram.
- README with setup and usage.
- Test/evaluation evidence.
- Security and limitations section.
- Short case study describing the problem and measurable result.